

A Comprehensive Mathematics Syllabus for Competitive Programming: Core Concepts, Complexity, and Reference Guide

Abstract

This document presents a comprehensive mathematics syllabus for competitive programming, serving as a checklist and reference guide. We cover essential topics across seven modules: (1) Number Theory (sieves, divisibility, modular arithmetic, and multiplicative inverses); (2) Combinatorics (binomial coefficients, factorials, Pigeonhole Principle, Catalan numbers, and Inclusion-Exclusion); (3) Algebra (linear recurrences, matrix exponentiation, and Fast Fourier Transform); (4) Probability and Game Theory (expected values, Nim-sums, and Sprague-Grundy theorem); (5) Bit Manipulation (submask iteration and built-in functions); (6) Computational Geometry (vectors, orientation, and convex hulls); and (7) Numerical Methods (ternary search and binary search on real numbers). For each module, we outline the core concepts, time complexities, and typical applications in competitive coding platforms.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis



1 Introduction

Mathematics is a key pillar of competitive programming (CP). A significant portion of problems on platforms such as Codeforces, CodeChef, AtCoder, and LeetCode require mathematical insights to reduce search spaces, optimize time complexities, or prove the correctness of greedy and dynamic programming solutions.

This syllabus provides a structured roadmap of the mathematical concepts required to excel in competitive programming, ranging from division and modulo operations to advanced algebraic and game-theoretic topics. It is designed to serve as a checklist and study guide for programmers seeking to elevate their performance from intermediate to advanced levels (e.g., reaching Candidate Master on Codeforces and beyond).

2 Module 1: Number Theory (The CP Core)

Number Theory is the most frequently tested mathematical area in competitive programming. Mastery of modular arithmetic, prime numbers, and divisibility is essential for solving division-based constraints.

2.1 Primality and Sieves

- Primality Testing: Naive $O(\sqrt{N})$ trial division and the probabilistic Miller-Rabin Primality Test ($O(K \log^3 N)$) for handling 64-bit integers ($N \leq 10^{18}$).
- Sieve of Eratosthenes: Standard $O(N \log \log N)$ algorithm to find all primes up to N .
- Sieve Optimizations: Odd-only sieving (reducing space by 50%), Linear Sieve (Euler's Sieve) which runs in $O(N)$ and computes the smallest prime factor (SPF) for every number (useful for $O(\log N)$ prime factorization queries).
- Segmented Sieve: Running sieves in blocks of size $O(\sqrt{N})$ to achieve $O(\sqrt{N})$ space complexity, bypassing memory limits for $N \leq 10^9$.

2.2 Divisibility and Greatest Common Divisor (GCD)

- Euclidean Algorithm: Computes $\gcd(a, b)$ in $O(\log(\min(a, b)))$ time.
- Extended Euclidean Algorithm: Solves the linear equation $ax + by = \gcd(a, b)$, yielding the coefficients x and y (useful for computing modular inverses).
- Linear Diophantine Equations: Solving $ax + by = c$. A integer solution exists if and only if $\gcd(a, b) \mid c$.

2.3 Modular Arithmetic

- Basic Operations: Addition, subtraction, and multiplication under a modulo M (typically $10^9 + 7$ or 998244353).
- Binary Exponentiation: Computing $a^b \pmod{M}$ in $O(\log b)$ time by recursively squaring the base.
- Modular Multiplicative Inverse: Finding x such that $ax \equiv 1 \pmod{M}$.
 - Via Fermat's Little Theorem: $a^{-1} \equiv a^{M-2} \pmod{M}$ (valid only when M is prime).
 - Via Extended Euclidean Algorithm: Finding x in $ax + My = 1$ (valid when $\gcd(a, M) = 1$, even if M is composite).

2.4 Advanced Number Theory

- Euler's Totient Function ($\phi(N)$): Counts the number of integers in $[1, N]$ that are coprime to N . Computed in $O(\sqrt{N})$ for a single query or $O(N)$ using a sieve.
- Euler's Theorem: $a^{\phi(M)} \equiv 1 \pmod{M}$ for $\gcd(a, M) = 1$. Offers a general method for exponent reduction under composite moduli.
- Chinese Remainder Theorem (CRT): Resolves a system of simultaneous congruences $x \equiv a_i \pmod{m_i}$ for pairwise coprime moduli in $O(k \log(\min(m_i)))$ time.
- Möbius Inversion & Dirichlet Convolution: Advanced tools for counting coprimes and optimizing nested loops on divisors.

3 Module 2: Combinatorics & Counting

Combinatorics is the study of counting discrete configurations. CP problems often ask for the count of valid states modulo M .

3.1 Factorials & Binomial Coefficients

- Permutations (nP_r) and Combinations (nC_r): Standard selection and arrangement formulas.
- Pre-computed Factorials: Computing factorials and their modular inverses up to $N \approx 10^6$ in $O(N)$ time. This enables $O(1)$ query response for ${}^nC_r \pmod{M}$.
- Pascal's Triangle: Generating binomial coefficients in $O(N^2)$ using additive DP (useful when modulo is not prime and division is impossible).
- Lucas Theorem: Computes $\binom{n}{r} \pmod{P}$ for a small prime P ($P \leq 10^5$) but large $n, r \leq 10^{18}$ by expressing n and r in base P .

3.2 Counting Principles & Identities

- Pigeonhole Principle (PHP): If n items are put into m containers where $n > m$, at least one container must contain more than one item. Used for proving invariants and cycles.
- Inclusion-Exclusion Principle (PIE): Computes the size of the union of overlapping sets. General formula:

$$\left| \bigcup_{i=1}^n S_i \right| = \sum_{\emptyset \neq J \subseteq \{1, \dots, n\}} (-1)^{|J|-1} \left| \bigcap_{j \in J} S_j \right|$$

- Catalan Numbers (C_n): Sequences satisfying $C_n = \frac{1}{n+1} \binom{2n}{n}$. Appears in counting:
 - Balanced parentheses sequences of length $2n$.
 - Valid Dyck paths on a grid.
 - Triangulations of a convex polygon with $n + 2$ sides.
 - Unlabeled binary trees with n nodes.
- Stirling Numbers: Stirling numbers of the second kind, $S(n, k)$, representing the number of ways to partition a set of n elements into k non-empty subsets.

4 Module 3: Algebra & Recurrences

Algebra in CP focused heavily on solving recurrence relations and polynomial operations.

4.1 Linear Recurrence & Matrix Exponentiation

- Fibonacci and Tribonacci recurrences: Solving state-transitions where $F_n = aF_{n-1} + bF_{n-2}$.
- Matrix Exponentiation: Represents a linear recurrence as a matrix multiplication $\mathbf{v}_n = \mathbf{M}\mathbf{v}_{n-1}$. Raising $\mathbf{M}$ to the power N in $O(D^3 \log N)$ (where D is the matrix dimension) allows computing the N -th term of a recurrence for $N \leq 10^{18}$.

4.2 Polynomials & Fast Fourier Transform (FFT)

- Polynomial Multiplication: Multiplying two polynomials of degree N naively takes $O(N^2)$ time.
- Fast Fourier Transform (FFT): Uses complex roots of unity to multiply polynomials in $O(N \log N)$ time.
- Number Theoretic Transform (NTT): A variant of FFT that operates over finite fields using primitive roots modulo M (commonly $M = 998244353$), avoiding floating-point precision errors.

5 Module 4: Probability & Game Theory

Many advanced CP problems involve expected values or finding optimal strategies in two-player games.

5.1 Probability & Expectation

- Conditional Probability & Bayes' Theorem: Computing probabilities of dependent events.
- Linearity of Expectation: For any random variables X and Y (even if dependent), $E[X + Y] = E[X] + E[Y]$. This property is used to break down complex expected value problems into simpler indicator variables.
- Probability Dynamic Programming: Defining states as probability distributions and transitioning between them.

5.2 Combinatorial Game Theory

- Impartial Games: Two-player games where valid moves depend only on the position, not on the player. Played under normal play convention (last player to move wins).
- Nim Game: The foundation of impartial games. A state is a winning position if and only if the XOR sum of the pile sizes (Nim-sum) is non-zero:

$$X = p_1 \oplus p_2 \oplus \dots \oplus p_k \neq 0$$

- Sprague-Grundy Theorem: Every impartial game is equivalent to a single Nim pile. The state is represented by a Grundy value (or SG number) computed recursively as the Minimum Excluded value (mex) of the Grundy values of all reachable states:

$$g(v) = \text{mex}(\{g(u) \mid v \rightarrow u\})$$

6 Module 5: Bit Manipulation & Masking

Bitwise operations provide high performance and compact state representations.

6.1 Basic Bitwise Operations

- Operators: AND (&), OR (|), XOR (^), NOT (~), Left Shift («), and Right Shift (»).
- Subsets as Bitmasks: Representing a subset of a set of size N as an integer in $[0, 2^N - 1]$. Checking membership via bitwise AND: $(\text{mask} \& (1 \ll i))$.

6.2 Advanced Masking Techniques

- Submask Iteration: Iterating through all submasks of a given mask in $O(3^N)$ total time:

```
for (int mask = 0; mask < (1 << N); mask++) {
    for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
        // Process submask
    }
}
```

- Built-in Functions (GCC):
 - `__builtin_popcount(x)`: Counts set bits.
 - `__builtin_clz(x)`: Counts leading zeros.
 - `__builtin_ctz(x)`: Counts trailing zeros.

7 Module 6: Computational Geometry

Geometry problems are less frequent but present a high barrier due to precision issues and edge cases.

7.1 Vector Arithmetic & Orientation

- Dot Product: $\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y$. Used to check perpendicularity ($\vec{a} \cdot \vec{b} = 0$) and project vectors.
- Cross Product: $\vec{a} \times \vec{b} = a_x b_y - a_y b_x$. Represents the signed area of the parallelogram formed by the vectors.
- Orientation Test (CCW): For three points A , B , and C , computing the cross product of $\vec{AB}$ and $\vec{BC}$:
 - Positive: Counter-Clockwise turn.
 - Negative: Clockwise turn.
 - Zero: Collinear points.

7.2 Geometric Structures & Algorithms

- Line Segment Intersection: Checking whether two segments intersect using orientation tests.
- Shoelace Formula: Computes the area of a polygon with vertices (x_i, y_i) in $O(N)$ time:

$$\text{Area} = \frac{1}{2} \left| \sum_{i=1}^n (x_i y_{i+1} - x_{i+1} y_i) \right|$$

- Convex Hull: Graham Scan or Monotone Chain algorithm to construct the convex boundary of N points in $O(N \log N)$ time.

8 Module 7: Numerical Methods & Optimization

8.1 Ternary Search

- Used to find the minimum or maximum of a unimodal function (functions that strictly increase then decrease, or vice-versa).
- Splits the search interval $[L, R]$ into three equal parts using two midpoints: $m_1 = L + \frac{R-L}{3}$ and $m_2 = R - \frac{R-L}{3}$.
- Runs in $O(\log N)$ and can be applied to both integer and floating-point ranges.

8.2 Binary Search on Answers (Real Numbers)

- Optimizing real-valued parameters by checking feasibility. Instead of a fixed iteration limit, run for a constant number of loops (typically 100 iterations) to guarantee high precision:

```
double L = 0.0, R = 1e9;
for (int iter = 0; iter < 100; iter++) {
    double mid = L + (R - L) / 2.0;
    if (check(mid)) L = mid;
    else R = mid;
}
```

9 Conclusion

Achieving proficiency in competitive programming mathematics requires a balanced approach. Programmers should begin by mastering modular arithmetic and the Sieve of Eratosthenes in Number Theory, and basic permutations and selections in Combinatorics. Once these foundational concepts are solidified, they can progress to more advanced topics such as matrix exponentiation, game theory (Sprague-Grundy), and Fast Fourier Transform (FFT). Applying these mathematical models to actual problem sets is the key to developing the necessary intuition for competitive coding assessments.